

Optimization and Uncertainty in Learning Report

student

I. INTRODUCTION

After a complete and thorough analysis of different classification techniques on the Adult and Wine Quality datasets presented in the SL report, this report aims to provide a comprehensive analysis of neural network optimization on these same datasets. The goal is to evaluate the specific behaviors of different optimization techniques and how they impact the final output. We will measure the potential improvements achievable relative to the results obtained in the SL Report and investigate the underlying reasons related to our design choices, dataset specificities, and algorithm behaviors.

Part 1 will focus on Randomized Optimization applied to the last network layers for both datasets to analyze whether these search-based algorithms can improve upon our previous performance. Part 2 will compare SGD and Adam ablations exclusively on the Adult dataset to analyze the specific behaviors of each component during training, as well as their impact on final performance and generalization. Finally, Part 3 will compare different regularization techniques on the same backbone using the Adult dataset to better understand and measure their specific behaviors and how they impact the optimization process under a fixed computational budget and optimizer.

II. HYPOTHESIS

To provide a meaningful analysis of how these optimization techniques impact learning and performance, we formulate and test several hypotheses grounded in our Exploratory Data Analysis (EDA) results, theoretical foundations, and existing literature on algorithm behavior.

A. Part 1 - Randomized optimization (RO) on the last several layers

For both datasets, we hypothesize that the Genetic Algorithm (GA) will perform best. Its population-based search explores broader regions of the parameter space, increasing the probability of escaping local optima. However, GA performance is likely constrained by the computational budget. Each step (pyperch) requires $1 + (n \times 2 + 1) \times (l + b)$ function evaluations (where n is the population size and l (resp. b) the number of unfrozen layers of weights (resp. bias)), potentially limiting its efficacy. We expect Simulated Annealing (SA) and Randomized Hill Climbing (RHC) to perform similarly provided that they are appropriately tuned to balance exploration and exploitation. For SA, an optimal temperature and cooling schedule may facilitate escaping local minima by temporarily accepting worse solutions. Similarly, RHC can achieve comparable results using an appropriate step size for fast, greedy convergence, coupled

with a restart policy to redirect search when improvements stagnate. On the Adult dataset, the medium feature count (42 post-preprocessing) and high sparsity (due to one-hot encoding and predominantly zero-valued *capital-loss* and *capital-gain* features) likely create a loss landscape with flat regions making convergence more difficult [1]. For the Wine dataset, severe class imbalance may generate a loss landscape with deep local minima, making it significantly harder for SA and RHC to efficiently escape exploitation and explore broader regions. Finally, we expect that none of the RO optimizers will surpass the SL baseline on the test set, although GA will likely achieve the closest performance.

B. Part 2 - Adam ablations on the full network

This section focuses solely on the Adult dataset across all optimizers. We hypothesize that standard SGD will perform adequately as our SL baseline, but adding momentum should reduce optimization zigzagging and facilitate escape from local minima. Momentum is also expected to accelerate convergence, especially across flat plateaus caused by dataset sparsity, leading to better local optima. Consequently, SGD with momentum and Nesterov accelerated gradient should converge more stably, yielding a narrower IQR across random seeds. Furthermore, we hypothesize that Adam variants will outperform SGD and Nesterov in both robustness and convergence speed. Adam's per-parameter adaptive learning rates, derived from the first and second moments, should enhance both convergence rates and seed robustness. Removing bias correction from Adam may induce instability during early optimization stages, as bias correction normally prevents initial steps toward zero [2]. Evaluating this behavior is an objective for this part. An RMSProp-like variant (Adam without the first moment/momentum) may still adapt to sparse features via the second moment, but we hypothesize it will be less stable than standard Adam. Finally, AdamW is expected to exhibit distinct behavior. Indeed, because weight decay is decoupled from the gradient update, using a fixed weight decay value (across all Adam variants) for fair comparison may impact AdamW differently than standard Adam, altering its relative performance. Finally, we hypothesize that this ablation study will yield better test set performance than our SL baseline due to more stable optimization across seeds, resulting in improved median results.

C. Part 3 - Regularization study

We hypothesize that while each regularization technique will uniquely affect the optimization process, all should reduce the generalization gap compared to the unregularized Adam baseline. L2 weight decay is expected to slow

convergence but mitigate overfitting, yielding better test-set performance. Early stopping may yield a higher validation loss than L2 regularization. It follows the standard Adam trajectory initially and halts before severe overfitting, but its efficacy relies heavily on the patience and tolerance hyperparameters. Dropout is anticipated to be the strongest regularizer by preventing complex co-adaptations, thereby improving generalization [3]. However, it may introduce noise into the loss curves, and high dropout rates could cause underfitting. For noise regularization we hypothesize that injecting noise will prevent the model from being overconfident across all class equally rather than specifically targeting the overrepresented class (≤ 50) [4], this smoothing of the decision boundary may improve generalization and the F1 score. Finally, we hypothesize that an optimal regularization strategy might combine noise injection (to improve calibration) with dropout or L2 weight decay (to prevent overfitting). Nevertheless, simply combining the independently best-tuned hyperparameters for each technique may not yield the best overall results, as regularization methods may interact in non-additive ways. Finally, we hypothesize that the best-performing configurations in this section will improve overall results on the test split, specifically due to a narrower generalization gap compared to our SL baseline.

III. DATA, METHODS AND SPLITS

A. Dataset preprocessing

Consistent with the SL Report, we use the Adult and Wine datasets. All experiments are conducted on a laptop equipped with a 2.30 GHz Intel i7-11800H CPU and 16 GB of RAM, without GPU acceleration. The following preprocessing and Exploratory Data Analysis details serve as a brief reminder, as they remain consistent with the previous report.

The Adult dataset contains information on 45,222 individuals. The task is binary classification, predicting whether annual income exceeds 50k. The target variable exhibits moderate imbalance, with 24.8% earning $> 50k$. The original dataset includes 6 quantitative and 7 qualitative features. EDA revealed high cardinality in categorical variables (e.g., *native-country* with 41 distinct, mostly underrepresented values), alongside skewness and sparsity in specific numerical variables. To address this, we one-hot encoded six categorical features with manageable cardinality. To prevent excessive sparsity and data leakage associated with target encoding, *native-country* was frequency encoded. A Yeo-Johnson transformation was applied to *age* and *fnlwt* (retained after analyzing non-linear relationships with a Random Forest classifier) to mitigate skewness and approximate a Gaussian distribution. Despite high sparsity, a $\log_1 p$ transformation was applied to *capital-gain* and *capital-loss* to stabilize variance and address skewness. All quantitative features were subsequently standardized. Finally, 47 duplicate rows and the redundant *education* column were removed, resulting in a final dataset of 45,175 rows and 42 features.

The Wine dataset contains information on 6,497 wines. The task is multiclass classification, predicting wine quality on a scale from 1 to 8. We removed 1,177 duplicate rows

to prevent data leakage and eliminate redundant information. The dataset consists of 11 continuous features and 1 binary-encoded variable. The primary challenge is severe target imbalance, with four classes (1, 2, 7, and 8) representing only 6% of total observations. Distribution curves and skewness coefficients (≥ 0.47) indicate significant right-skewness across most features. To correct this, we applied Yeo-Johnson transformations to all numerical features except *pH* and *total_sulfur_dioxide*, which did not exhibit strong skewness. All quantitative features were subsequently standardized. The final preprocessed dataset comprises 5,320 rows and 12 features.

For data splitting, we opted for a hold-out validation approach with a final distribution of 20% testing, 16% validation, and 64% training. Due to class imbalance in both datasets, we utilized stratified splitting to maintain consistent target distributions and ensure representation of all classes across splits. The validation set is used for hyperparameter tuning and plotting, the training set for model fitting, and the test set exclusively for evaluating final metrics using the optimal hyperparameters. Data splits remain consistent across all experiments and were generated using seed 4. While the SL Report utilized cross-validation, we chose a hold-out approach to facilitate the multi-seed analysis required to evaluate stability across randomness using the median and IQR of all metrics. Combining cross-validation with multiple seeds would have introduced unnecessary computational overhead, and course staff confirmed that combining a hold-out validation split with multiple seeds is reasonable. To ensure reproducibility across all multi-seed experiments, algorithmic randomness was controlled using five specific seeds: [4, 44, 444, 4444, 44444]. Additionally, all models were evaluated deterministically using `model.eval()`.

B. Methodology

To compare performance across all optimization methods on the test set, we decided to use the F1 score for the Adult dataset, as it is a powerful metric for addressing class imbalance. Similarly, we use the Macro-F1 score for the Wine dataset to handle the same issue in a multiclass setting. Across all parts, we consistently report the results of the best model by saving the checkpoint that achieves the lowest validation loss within our optimization budget.

To maintain consistency with our previous SL report, we use the same fixed PyTorch backbone architectures for all subsequent experiments with standard PyTorch initialization (deterministic for each seed) for Parts 2-3. For the Wine dataset, the deep backbone performed best, so we retain it (10,652 parameters). For the Adult dataset, the shallow version performed slightly better, but we chose the deep version (11,341 parameters) to maintain consistency with the Wine dataset. This decision also allows us to unfreeze more than just the classification layer (typically hidden layers) in Part 1, enabling a clearer observation of random optimizer impacts without arbitrarily adding layers to the shallow version. Furthermore, the validation F1 score difference between the

shallow and deep versions was only 0.01. Both deep backbones are simple MLPs with three hidden layers and ReLU activations, using a sigmoid activation with binary cross-entropy for Adult, and a softmax activation with categorical cross-entropy for Wine. To obtain comparable metrics, we retrain using standard SGD without momentum and evaluate our baselines across the same five random seeds to ensure unbiased comparisons against the evaluated methods (Tab. I). Following the previous report, we consistently use batch sizes of 128 for Adult and 32 for Wine (except for part 1, where each function evaluation is performed on the entire dataset at once).

TABLE I
BASELINE RESULTS ON TEST SPLIT (MEDIAN $\pm$ IQR)

	Best val loss	Test F1/Macro-F1	Test Accuracy	hidden layer sizes	Hyperparameters
Adult	0.325 $\pm$ 0.004	0.670 $\pm$ 0.010	0.848 $\pm$ 0.002	[50, 90, 50]	learning rate=0.1 L2 weight decay = 0.001 ES patience = 5 (tol=1e-4)
Wine	1.069 $\pm$ 0.009	0.294 $\pm$ 0.052	0.545 $\pm$ 0.008	[60, 80, 56]	learning rate=0.1 L2 weight decay = 0.0001 ES patience = 5 (tol=1e-4)

We define identical budgets across all methods and parts to ensure consistent evaluation and fair algorithmic comparisons. Wall-clock time, function evaluation limits, and validation-loss thresholds ℓ were determined by testing standard parameters in Parts 1 and 2 to observe convergence and plateauing behavior, and a ℓ reachable by the majority of algorithms (Tab. II). The gradient evaluation budget was fixed to match our SL backbone training process (50 epochs) using the following formula: $n_epoch \times \text{ceil} \left(\frac{\text{train_size}}{\text{batch_size}} \right)$. All optimization methods are stopped once the budget is reached.

TABLE II
BUDGET AND THRESHOLD VALUES

	Gradient evaluations	Function evaluations	Wall-clock (s)	Validation-loss threshold ℓ
Adult	11300	25000	300	0.318
Wine	-	25000	300	1.0

For each section, we employ an algorithm-specific methodology, detailed below.

1) *Part 1 - Randomized optimization (RO)*: In this part, we utilize the *pyperch* library to implement three randomized optimization algorithms: Randomized Hill Climbing, Simulated Annealing, and Genetic Algorithm. Across all algorithms and datasets, we freeze the first layers and optimize the final two (the last hidden layer and the output layer). Initial weights are derived from a baseline optimization using seed 4, serving as the initialization (warm start) for both frozen and unfrozen layers. This setup results in 4,601 trainable parameters (out of 11,341 total) for the Adult dataset and 4,992 (out of 10,652 total) for the Wine dataset. Validation loss serves as the fitness objective.

Because the *pyperch* implementation of RHC lacks a restart policy, we implement our own by introducing three new parameters alongside the default *step_size*. We tested multiple values within the following ranges: *step_size*

[0.01, 0.1], *patience* [50, 200] (the number of iterations without improvement before restarting), *tolerance* [10^{-6} , 10^{-3}] (the minimum improvement threshold), and *noise_scale* [0.1, 1] (the standard deviation of a centered Gaussian noise added to the pretrained weights to vary initialization upon restart). For SA, we tuned the standard *pyperch* parameters across defined ranges: *step_size* [0.01, 0.1], initial temperature [10, 10000], minimal temperature [10^{-6} , 10^{-4}], and cooling decay [0.8, 0.99]. For GA, we explored various values for population size [20, 100], mutation rate [0.1, 0.5], and *step_size* [0.005, 0.1]. The selection mechanism employs roulette wheel selection, assigning weights inversely proportional to the loss to favor individuals with lower losses. Uniform crossover mixes bits across half the population. The mutation rate defines the proportion of individuals mutated per iteration. Finally, we employ elitism alongside roulette wheel selection. If the new population’s best loss fails to improve upon the previous step, the entire previous population is restored.

Following the *pyperch* specifications, perturbations are applied using a single global scale across all unfrozen layers, without layer-aware scaling or weight clipping. The weight perturbation for SA and RHC follows a uniform distribution between $-\frac{\text{step_size}}{2}$ and $\frac{\text{step_size}}{2}$ using a constant adaptation schedule. For GA initialization and mutation, we apply a centered Gaussian distribution with a standard deviation equal to *step_size*, also maintaining a constant adaptation schedule.

2) *Part 2 - Adam ablations on the full network*: In this part, we compare standard gradient-based methods (SGD, momentum, Nesterov) with adaptive approaches (Adam ablations) exclusively on the Adult dataset. To ensure a fair and consistent comparison with our SL baseline results, the weight decay is fixed to the same 0.001 value (also for AdamW) without further tuning. Contrary to the SL baseline, we do not employ early stopping, as we aim to evaluate performance at a fixed computational budget and already select the best checkpoint during optimization for final test metrics. Hyperparameter tuning is conducted consistently across all seven methods using a grid search, retaining the hyperparameters that perform best across the five seeds. The learning rate $lr \in \{10^{-3.5}, 10^{-3}, 10^{-2.5}, 10^{-2}, 10^{-1.5}, 10^{-1}\}$ is tuned for all methods. Additionally, the momentum parameter $momentum \in \{0.8, 0.9, 0.99\}$ is tuned for SGD with momentum and Nesterov, while $\beta_1 \in \{0.8, 0.9, 0.99\}$ (except for the RMSProp-like variant) and $\beta_2 \in \{0.9, 0.99, 0.999\}$ are tuned alongside the learning rate for the Adam family. Adam without bias correction was implemented manually, as the PyTorch implementation does not provide an option to disable this correction.

3) *Part 3 - Regularization study*: Here, we measure the impact of regularization on performance and generalization by evaluating four techniques: L2 weight decay, early stopping, dropout (applied after each ReLU activation in the hidden layers), and noise regularization. To better assess their impact, we establish a baseline trained without any regularization (removing even the weight decay from Part

2), using the optimal Adam hyperparameters identified previously. We independently tune the hyperparameters for each regularization technique across defined grids: the penalty value for weight decay in $\{10^{-5}, 10^{-4.5}, \dots, 10^{-2}\}$; patience in $\{1, 3, 5, 10\}$ and tolerance in $\{10^{-5}, 10^{-3}, 10^{-1}\}$ for early stopping; label smoothing in $\{0.001, 0.01, 0.1\}$ and the standard deviation of centered Gaussian input noise in $\{0.01, 0.05, 0.1, 0.2\}$ for noise regularization; and the dropout probability in $\{0.1, 0.2, \dots, 1.0\}$. Finally, we search for the best combination of at least two techniques using their independently optimized hyperparameters.

IV. RESULTS

A. Part 1 - Randomized optimization (RO)

1) *Adult dataset*: For the Adult dataset, the RO results are mixed. In terms of the best validation loss, SA achieved the lowest value (0.313, see Tab. IV), followed by RHC and GA. However, this trend reverses for the test metrics (F1 score and accuracy), where GA performs best. This discrepancy stems from the difference between the continuous fitness objective (cross-entropy loss) and discrete evaluation metrics. The loss penalizes low prediction confidence based on output probabilities, whereas F1 and accuracy evaluate the discrete decision boundary. Consequently, a lower loss may simply reflect increased confidence on already correctly classified examples without improving overall classification accuracy. Furthermore, as hypothesized, the sparsity-induced flat regions of the Adult loss landscape [1] likely trap the greedy RHC and SA in sharp, poorly generalizing local minima. In contrast, GA’s population-based search effectively navigates these plateaus to find flatter optima that generalize better to the unseen test set.

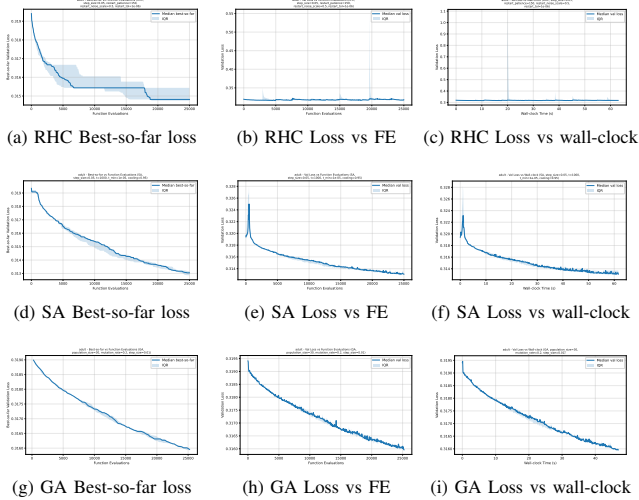


Fig. 1. RO progress and loss vs compute curves for Adult Dataset

Regarding computation and runtime, RHC’s best configuration (see Fig. 1 for exact hyperparameters) achieves the fastest convergence to the fixed ℓ threshold in just 1.6s, compared to 4.1s for SA and 8.8s for GA. This is driven by RHC’s greedy approach, which rapidly descends into the nearest local minimum, often finding a good solution within

the first few function evaluations (684). Figure 1 illustrates this with RHC’s best-so-far loss periodically plateauing. When a local minimum is reached, a restart occurs, visible as sharp spikes in the loss-vs-compute curves due to noisy re-initialization. Conversely, SA avoids restarts and balances exploration and exploitation using a temperature schedule. High initial temperatures ($T = 1000$) promote early exploration, visible as early spikes and a flat best-so-far curve in Fig. 1, before progressively cooling ($t_{min} = 10^{-5}$) to exploit a local minimum, ultimately yielding a lower validation loss than RHC. Unlike the other two algorithms, GA exhibits smooth, stable optimization curves with tight IQR bands. This stability, coupled with an absence of plateauing at the end of the 25,000 function evaluations, suggests GA was bottlenecked by the budget. Its population-based updates require significantly more evaluations per step. Despite identical function evaluation budgets, GA completes its total run faster than SA and RHC. This efficiency gap stems from algorithmic overhead: local search methods perform only two evaluations per step (requiring 12,500 Python loop iterations), whereas GA batches evaluations across the population, exhausting the budget in 103 steps. Furthermore, inspecting the *pyperch* source code reveals that RHC and SA generate perturbation noise via NumPy before moving it to PyTorch at every step, creating a bottleneck compared to GA, which generates noise simultaneously for the entire population directly in PyTorch.

Finally, these results confirm our hypothesis regarding GA’s superiority on test metrics for the Adult dataset, despite budget constraints hindering deeper optimization. They also highlight that carefully balancing exploration and exploitation (via SA’s temperature schedule or RHC’s restart policy) is crucial for strong performance. However, our initial hypothesis that RO would not surpass the SL baseline must be moderated: both RHC and GA achieved higher F1 scores than the baseline, demonstrating the usefulness of RO for fine-tuning a partially trained backbone.

2) *Wine dataset*: The RO results on the Wine dataset challenge our initial hypotheses regarding relative algorithmic performance. On this dataset, the best-performing tuned algorithm is RHC (see Fig. 2 caption for hyperparameters), which achieves the best test F1 score, accuracy, validation loss, and time to the ℓ threshold. Moreover, RHC demonstrates significantly greater robustness across random seeds. Its IQR for function evaluations to reach the threshold is much tighter (222) compared to SA (6524) and GA (1470). While Figure 2 illustrates algorithmic trajectories broadly similar to those observed on the Adult dataset, the target loss threshold proved harder to reach, inducing greater overall variance. For RHC, the best-so-far curve indicates that restarts occur less frequently and are less efficient than on the Adult dataset. This is partly due to the tuned hyperparameters, specifically a slightly lower step size and a higher restart patience, while a lower noise scale yields less disruptive perturbations during restarts (Fig. 2b). For SA, the higher initial temperature ($T = 10000$) and slower cooling decay (0.99) extend the high-temperature exploration phase, resulting in a prominent

TABLE III
SUMMARY TABLE FOR WINE DATASET ACROSS 5 SEEDS (MEDIAN $\pm$ IQR)

Method	Best Validation Loss	Test F1	Test Accuracy	Time to ℓ in seconds (total runtime)	Gradient Evaluation to ℓ (total)	Function Evaluation to ℓ (total)	Updates to ℓ (total)
Randomized Hill Climbing	0.984 $\pm$ 0.003	0.361 $\pm$ 0.004	0.556 $\pm$ 0.002	6.8 $\pm$ 0.2 (28.7 $\pm$ 2.0)	-	5844 $\pm$ 222 (25000 $\pm$ 0)	2921 $\pm$ 111 (12500 $\pm$ 0)
Simulated Annealing	0.996 $\pm$ 0.005	0.352 $\pm$ 0.004	0.541 $\pm$ 0.002	22.1 $\pm$ 3.3 (25.3 $\pm$ 0.6)	-	21404 $\pm$ 6524 (25000 $\pm$ 0)	10701 $\pm$ 3262 (12500 $\pm$ 0)
Genetic Algorithm	0.996 $\pm$ 0.002	0.357 $\pm$ 0.003	0.549 $\pm$ 0.003	15.4 $\pm$ 0.7 (17.4 $\pm$ 0.3)	-	22295 $\pm$ 1470 (25000 $\pm$ 0)	90 $\pm$ 6 (103 $\pm$ 0)

initial plateau. This prolonged exploration ultimately fails to achieve a validation loss lower than RHC, likely because the algorithm was bottlenecked by the computational budget before it could fully exploit a minimum, as the final tail does not fully plateau. Consequently, SA exhibits high seed dependency, reflected in its massive FE IQR of 6524. Finally, GA yields moderate results, with its optimization curves mirroring the smooth, stable trends seen on the Adult dataset. However, the computational budget might have hindered its optimization. The lack of a final plateau confirms that GA’s heavy evaluation cost per update restricted its convergence. While GA benefits from the same PyTorch batching efficiency noted previously, yielding a lower total runtime, this computational advantage did not allow it to reach the ℓ threshold faster than the greedy RHC (15.4s vs. 6.8s median).

divergence is a direct consequence of severe class imbalance. Because accuracy heavily biases toward majority classes, RO algorithms likely sacrificed some majority-class predictions to correctly classify the harder, underrepresented minority classes. This adds nuance to our initial hypothesis regarding the Wine loss landscape. While standard SGD optimization converged to a local minimum dominated by majority-class predictions, the combined greedy and exploratory behaviors of RO found more optimal, deeper local minima. These minima may have mitigate the steep cross-entropy penalties associated with confidently wrong minority-class predictions, shifting the decision boundary to better capture underrepresented classes and thereby improving the F1 score.

B. Part 2 - Adam ablations on the full network

For the ablation study on the Adult dataset, all optimizers rapidly decrease their validation loss during the first 2000 gradient evaluations before stabilizing at a lower loss plateau. However, depending on the specific characteristics of each optimizer, our results convey several interesting insights regarding their behaviors.

As hypothesized, standard SGD (best LR of 0.1) behaves adequately compared to our SL baseline. However, despite using the same seeds, preprocessing, and hyperparameters, the results differ slightly because this ablation study omits early stopping. The baseline’s low patience yields a higher best validation loss (due to early termination) but a better F1 score, reinforcing that continuous loss does not scale linearly with discrete evaluation metrics. Adding momentum to SGD (best configuration: LR $10^{-1.5}$, momentum 0.8) not only outperforms standard SGD in stability (Fig. 4), exhibiting a narrower IQR and attenuating the zigzagging phenomena seen in the standard SGD curve, but also improves convergence speed. Table IV shows that SGD with momentum reaches the loss threshold ℓ in only 8.6s (versus 11.1s for standard SGD). This trend is consistent for GE, requiring approximately 3500 fewer evaluations with a smaller IQR. The Nesterov optimizer (LR 0.01, momentum 0.9) requires more time and GE to reach ℓ , but slightly reduces zigzagging. This slower convergence compared to SGD with momentum is likely due to the lower learning rate and the highly sparse Adult loss landscape, which induces micro-adjustments. Overall, while momentum-based SGD variants take slightly longer to exhaust the total budget (+3 to 4s), they reach ℓ faster in terms of GE. Furthermore, SGD with momentum is the fastest to reach ℓ in wall-clock time (8.6s), even when compared to Adam variants.

Regarding the Adam variants, our hypothesis predicting better stability compared to earlier optimizers is refuted on this dataset. The IQR bands and metrics (Fig. 4, Tab. IV) demonstrate that standard Adam fails to achieve an F1

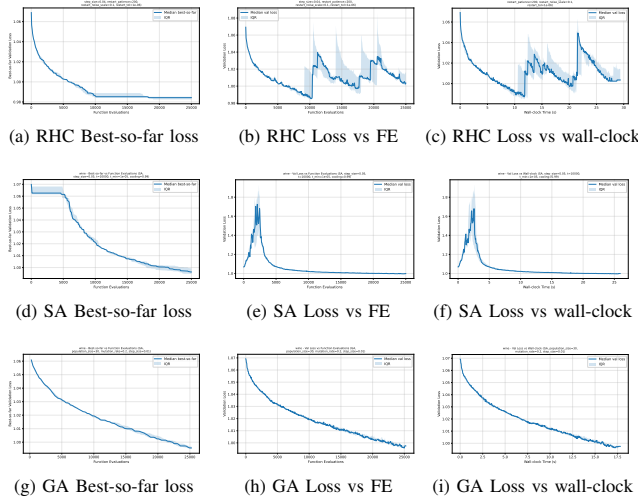


Fig. 2. RO progress and loss vs compute curves for Wine Dataset

Finally, contrary to our initial hypothesis regarding performance relative to the SL baseline, our results in Tab. III show that all RO algorithms surpass the baseline (Tab. I) in F1 score and almost always in test accuracy (with the exception of SA). This demonstrates that randomized optimization can be an effective method for fine-tuning pre-trained networks in highly imbalanced contexts. However, while RO systematically outperforms the SL baseline on the Wine dataset, this comparison must be nuanced. Table I reports aggregated cross-seed baseline metrics, but seed 4 (used for pre-trained weight initialization) actually achieved the best individual baseline performance (Test F1: 0.343, Accuracy: 0.559). This high baseline variance across seeds likely stems from aggressive early stopping and a high learning rate. Relative to this unique strong seed 4 baseline, RO algorithms achieved lower validation losses and higher Test F1 scores (e.g., RHC: 0.361 vs. 0.343) but slightly lower accuracy (e.g., RHC: 0.556 vs. 0.559). This metric

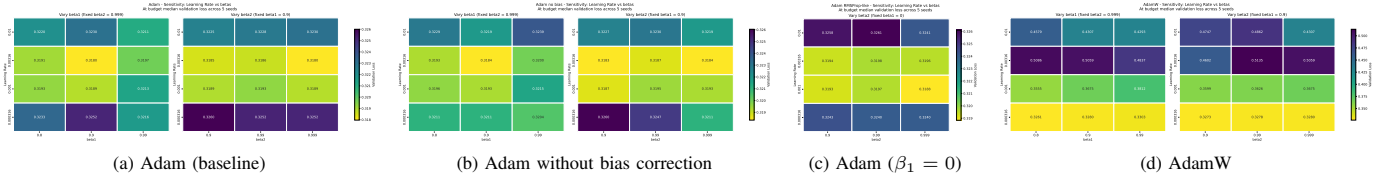


Fig. 3. Sensitivity heatmaps for Adam variants

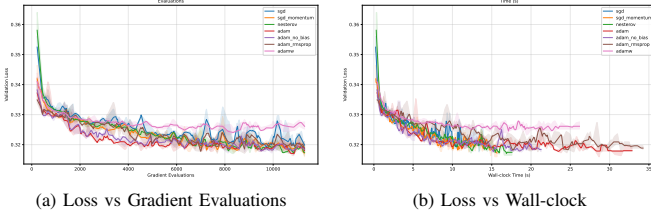


Fig. 4. Loss vs compute curves for optimizers (Median $\pm$ IQR)

score IQR lower than the 0.001 observed with SGD with momentum. Moreover, the stability bands in Fig. 5 indicate that momentum variants offer much narrower validation loss distributions than Adam variants. Our hypothesis regarding the faster convergence of the Adam family must also be nuanced. On one hand, standard Adam (best configuration: LR $10^{-2.5}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) requires the fewest GE to reach ℓ (median 5420), making it computationally efficient per step. On the other hand, SGD with momentum remains faster in wall-clock time to reach ℓ . This discrepancy is directly tied to the total runtime of each algorithm: Adam variants consistently run for over 20s, whereas the first three optimizers remain under 17s. This delay stems from the additional operations required by Adam, which slow down step execution. This operational overhead also explains why Nesterov and SGD with momentum take longer than standard SGD, as they introduce first-moment calculations. Similarly, Adam without bias correction (LR $10^{-2.5}$, $\beta_1 = 0.9$, $\beta_2 = 0.9$) is 8s faster than standard Adam due to the removal of heavy division and exponentiation operations.

Challenging our initial hypotheses, the results show that removing bias correction does not make the optimizer significantly less stable than standard Adam in early steps, as seen in the IQR shaded bands of Fig. 4. It does, however, alter convergence speed, requiring more GE to reach ℓ (7230 versus 5420) but with a drastically lower cross-seed IQR (226 versus 1360). This behavior can be attributed to the $\beta_2 = 0.9$ value selected by the grid search, which partially compensates for the lack of early bias correction by increasing the denominator, thereby forcing smaller initial update steps. This lower β_2 value also reduces Adam’s gradient memory, potentially improving stability by quickly forgetting anomalies caused by dataset sparsity. Furthermore, as hypothesized for the RMSProp-like Adam variant (learning rate 0.001, $\beta_2 = 0.999$), removing the momentum term negatively impacts validation loss stability (Fig. 5) and increases the median GE required to cross ℓ by 400. This makes it the slowest optimizer to reach the budget (32.3s), exacerbating standard Adam’s already slow runtime (28.5s). This phenomenon mirrors the contrast between standard SGD and SGD with momentum. However, in this case, the

isolated second moment term likely guides the optimizer to a minimum that generalizes better on test metrics, yielding the highest F1 score among the Adam variants.

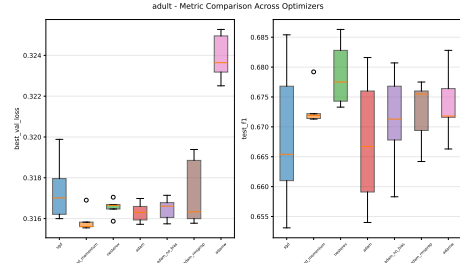


Fig. 5. Part 2 stability bands along 5 seeds for best validation loss/test F1

Furthermore, as hypothesized, Fig. 4 shows that AdamW (best configuration: learning rate $10^{-3.5}$, $\beta_1 = 0.8$, $\beta_2 = 0.999$) clearly overfits and behaves differently from other variants, never reaching the ℓ threshold across all tested hyperparameters. This overfitting is clearly measurable in Fig. 6, where its generalization gap is the highest among all optimizers (approximately 0.035 versus 0.015) and displays a steeper upward trajectory. This is likely due to the decoupled nature of its weight decay, rendering the fixed regularization value inappropriate compared to the other optimizers. However, this overfitting on the loss does not severely impact the F1 score, which remains quite competitive (even beating the Adam baseline F1), likely for the same continuous-versus-discrete metric reasons explained in Part 1. Figure 3 demonstrates that, similar to SGD and its variants, the Adam family is highly sensitive to learning rate variations (indicated by vertical banding) and much less sensitive to differences in β parameters. We observe that standard Adam and the bias-free variant display highly similar sensitivity patterns. The RMSProp-like variant also follows suit, though its optimal learning rate is shifted 1 lines downward, likely to compensate for the missing first-moment term. Finally, AdamW exhibits overall higher loss values and favors lower learning rates, further confirming that the chosen weight decay value was inadequate for this specific architecture and optimizer.

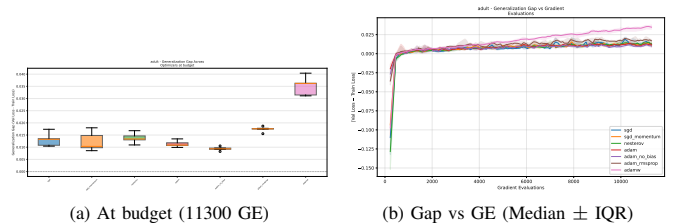


Fig. 6. Optimizers generalization gap curves. The initial negative gap occurs because train loss is averaged over the epoch, whereas validation loss is computed at the end with the latest, updated weights.

To conclude, we can accept our initial hypothesis regarding the superior performance of specific optimizers over the baseline. Despite most optimizers sharing a similar generalization gap, which is even narrower for Adam variants (excluding AdamW) (Fig. 6), our findings indicate that stability in the best validation loss and generalization gap across seeds does not necessarily translate to robust test set generalization, as highlighted by the stability bands in Fig. 5. In terms of runtime convergence speed and performance stability, SGD with momentum performs best. Conversely, Nesterov achieves the highest median F1 score with a comparatively low IQR (0.009). Among the Adam variants, the RMSProp-like optimizer yields the highest test F1 score. Ultimately, the default unaccelerated SGD and standard Adam optimizers produce the worst F1 scores on the test set, while also exhibiting the highest variance.

C. Part 3 - Regularization study

Building upon Part 2, the optimal hyperparameters fixed for Adam in this study are $LR=10^{-2.5}$, $\beta_1 = 0.9$, and $\beta_2 = 0.999$. This part analyzes how the standard Adam optimizer behaves with various regularization techniques under fixed constraints (budget and optimizer hyperparameters) to evaluate potential generalization improvements.

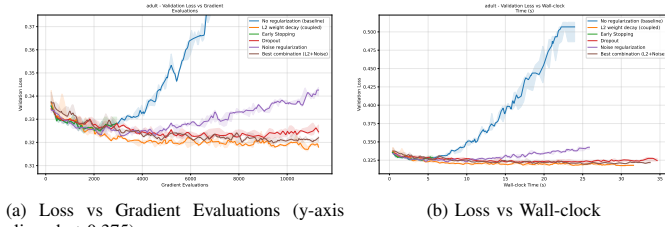


Fig. 7. Loss vs compute curves for regularization methods (Median $\pm$ IQR)

Contrary to our hypothesis that the best regularization would strictly improve test performance, our most effective single regularizer, noise regularization (best tuned hyperparameters: label smoothing=0.01, input noise=0.1), achieved the same median test F1 score but narrowed the IQR across seeds, improving stability (Tab. IV and Fig. 8). Thus, our hypothesis regarding noise regularization is only partially verified for F1 score generalization, though it is strongly supported regarding cross-seed stability.

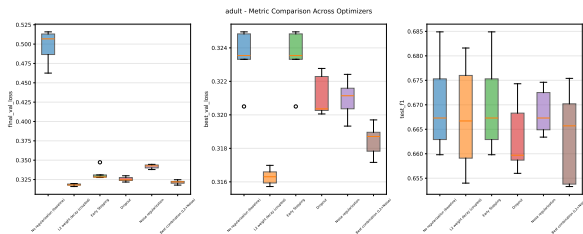


Fig. 8. Part 3 stability bands along 5 seeds for final/best validation loss and test F1

Figure 8 suggests that while test F1 and accuracy are not significantly improved by regularization, both the best and final validation losses (at budget) are reduced across all techniques, with L2 weight decay achieving the minimum. Loss

versus compute curves (Fig. 7) reinforce this tendency, showing the unregularized Adam baseline completely overfitting by the final gradient evaluation. Noise regularization exhibits the second-highest loss curves, indicating that low noise hyperparameters failed to completely prevent overfitting. Although regularization techniques achieved lower validation losses (prompting checkpoint selection for test evaluation), cross-entropy primarily penalizes confidence, whereas F1 evaluates the decision boundary. Regularization may have adjusted model confidence without shifting the boundary to capture misclassified elements. Furthermore, most techniques (except L2 and the best combination) failed to cross the ℓ threshold. This likely occurred because the optimal optimizer hyperparameters were tuned using Part 2’s fixed weight decay, making it difficult for other regularization methods to compensate for its removal. Figure 9 confirms our hypothesis that regularization reduces the generalization gap, mitigating the baseline’s overfitting. The persistent negative generalization gap for the best combination across the compute budget likely occurs because injected noise artificially inflates the training cross-entropy loss, while validation passes remain noise-free. This elevated training loss may also shift other regularizers’ generalization gap curves toward zero.

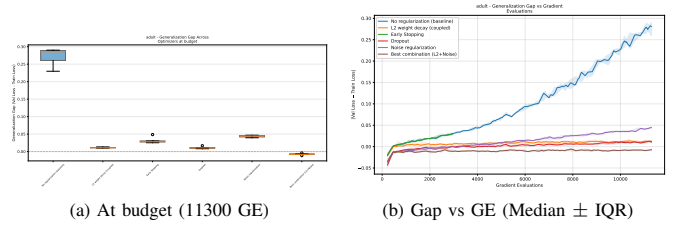


Fig. 9. Regularization methods generalization gap curves

Because we evaluate test results using the best optimization checkpoint, early stopping (tuned: patience=3, tol= 10^{-5}) yields the exact same test performance as unregularized Adam. The critical difference lies in the training window: while other techniques consume the entire budget before checkpointing, early stopping halts optimization once improvements fall below tolerance for three epochs. The primary benefit is a drastic reduction in computational time. As shown in Table IV, it stops after only 2938 gradient evaluations (compared to 11300). This contrast highlights a crucial inefficiency: unregularized Adam achieves its optimal generalization state within the first 24% of the allocated compute budget, effectively wasting the remaining 76% on overfitting the training data. This confirms our hypothesis regarding ES inferior validation loss compared to L2, trading optimal loss for computational efficiency when the ideal iteration count is unknown.

Contrary to our hypothesis, Dropout (tuned: rate=0.4) degraded test F1 performance (0.660 vs. 0.667). In this highly sparse dataset, dropout may randomly zero out already rare information, slightly lowering performance. Future work could explore applying dropout only to specific layers. Meanwhile, L2 regularization (tuned: penalty=0.001) achieved a strong loss minimum but failed to translate this into a better F1 score than the baseline, while increasing the IQR

TABLE IV
SUMMARY TABLE FOR ADULT DATASET ACROSS 5 SEEDS (MEDIAN $\pm$ IQR)

Method	Best Validation Loss	Test F1	Test Accuracy	Time to ℓ in seconds (total runtime)	Gradient Evaluation to ℓ (total)	Function Evaluation to ℓ (total)	Updates to ℓ (total)
Randomized Hill Climbing	0.315 $\pm$ 0.000	0.671 $\pm$ 0.003	0.849 $\pm$ 0.001	1.6 $\pm$ 0.8 (58 $\pm$ 7)	-	684 $\pm$ 276 (25000 $\pm$ 0)	341 $\pm$ 138 (12500 $\pm$ 0)
Simulated Annealing	0.313 $\pm$ 0.000	0.668 $\pm$ 0.002	0.849 $\pm$ 0.001	4.1 $\pm$ 0.6 (59 $\pm$ 5)	-	1720 $\pm$ 188 (25000 $\pm$ 0)	860 $\pm$ 94 (12500 $\pm$ 0)
Genetic Algorithm	0.316 $\pm$ 0.000	0.673 $\pm$ 0.003	0.851 $\pm$ 0.001	8.8 $\pm$ 1.1 (46 $\pm$ 2)	-	4900 $\pm$ 245 (25000 $\pm$ 0)	19 $\pm$ 1.0 (103 $\pm$ 0)
SGD (no momentum)	0.317 $\pm$ 0.002	0.665 $\pm$ 0.016	0.850 $\pm$ 0.001	11.1 $\pm$ 2.5 (12.2 $\pm$ 0.6)	10200 $\pm$ 1750 (11300 $\pm$ 0)	-	10200 $\pm$ 1750 (11300 $\pm$ 0)
SGD (momentum)	0.316 $\pm$ 0.000	0.672 $\pm$ 0.001	0.851 $\pm$ 0.001	8.6 $\pm$ 1.7 (15.4 $\pm$ 0.8)	6550 $\pm$ 678 (11300 $\pm$ 0)	-	6550 $\pm$ 678 (11300 $\pm$ 0)
Nesterov momentum	0.317 $\pm$ 0.000	0.677 $\pm$ 0.009	0.851 $\pm$ 0.001	12.5 $\pm$ 1.4 (16.8 $\pm$ 0.3)	8140 $\pm$ 904 (11300 $\pm$ 0)	-	8140 $\pm$ 904 (11300 $\pm$ 0)
Adam (baseline)	0.316 $\pm$ 0.000	0.667 $\pm$ 0.017	0.849 $\pm$ 0.001	13.1 $\pm$ 4.2 (28.5 $\pm$ 1.5)	5420 $\pm$ 1360 (11300 $\pm$ 0)	-	5420 $\pm$ 1360 (11300 $\pm$ 0)
Adam (no bias correction)	0.317 $\pm$ 0.000	0.671 $\pm$ 0.009	0.849 $\pm$ 0.000	12.8 $\pm$ 0.9 (20.6 $\pm$ 0.7)	7230 $\pm$ 226 (11300 $\pm$ 0)	-	7230 $\pm$ 226 (11300 $\pm$ 0)
Adam ($\beta_1 = 0$)	0.316 $\pm$ 0.003	0.675 $\pm$ 0.007	0.851 $\pm$ 0.002	17.7 $\pm$ 2.6 (32.3 $\pm$ 0.9)	5880 $\pm$ 791 (11300 $\pm$ 0)	-	5880 $\pm$ 791 (11300 $\pm$ 0)
AdamW	0.324 $\pm$ 0.002	0.672 $\pm$ 0.005	0.849 $\pm$ 0.002	Never (23.6 $\pm$ 0.3)	Never (11300 $\pm$ 0)	-	Never (11300 $\pm$ 0)
Adam (No regularization)	0.324 $\pm$ 0.002	0.667 $\pm$ 0.012	0.850 $\pm$ 0.003	Never (23.0 $\pm$ 1.0)	Never (11300 $\pm$ 0)	-	Never (11300 $\pm$ 0)
L2 weight decay	0.316 $\pm$ 0.001	0.667 $\pm$ 0.017	0.849 $\pm$ 0.001	12.9 $\pm$ 4.0 (29.3 $\pm$ 0.9)	5420 $\pm$ 1356 (11300 $\pm$ 0)	-	5420 $\pm$ 1356 (11300 $\pm$ 0)
Early stopping	0.324 $\pm$ 0.002	0.667 $\pm$ 0.012	0.850 $\pm$ 0.003	Never (5.5 $\pm$ 0.1)	Never (2938 $\pm$ 226)	-	Never (2938 $\pm$ 226)
Dropout	0.320 $\pm$ 0.002	0.660 $\pm$ 0.010	0.849 $\pm$ 0.001	Never (32.9 $\pm$ 1.3)	Never (11300 $\pm$ 0)	-	Never (11300 $\pm$ 0)
Noise regularization (Best single)	0.321 $\pm$ 0.001	0.667 $\pm$ 0.008	0.850 $\pm$ 0.004	Never (25.1 $\pm$ 1.1)	Never (11300 $\pm$ 0)	-	Never (11300 $\pm$ 0)
Best combination (L2 + Noise)	0.319 $\pm$ 0.001	0.666 $\pm$ 0.016	0.849 $\pm$ 0.002	31.4 $\pm$ 2.1 (33.5 $\pm$ 2.2)	10961 $\pm$ 339 (11300 $\pm$ 0)	-	10961 $\pm$ 339 (11300 $\pm$ 0)

and reducing stability. As hypothesized, the combination of independently tuned hyperparameters (L2 + Noise) yielded the second-best validation loss in our study of combined methods. However, this combination yielded an inferior median test F1 score of 0.666 with a high IQR of 0.016. These results confirm that regularization interactions may be non-additive and need to be tuned simultaneously for better optimization of combined performance.

In conclusion, contrary to our initial hypotheses, Part 3 did not identify a regularization technique that improved the SL baseline’s test F1 score. However, because the loss metrics were successfully reduced, further analysis could investigate adjusting the classification threshold to translate these lower losses into improved F1 test performance.

V. CONCLUSION AND DISCUSSION

The primary objective of this report was to move beyond treating neural networks as black boxes by identifying deterministic, causal relationships between specific training configurations and their outcomes. By enforcing strict budget-matched evaluations and analyzing dispersion across fixed seeds, we evaluated interventions against our SL baseline to establish clear practitioner guidelines. Across our experiments, interventions yielded mixed success relative to the SL Report baseline.

Randomized Optimization consistently improved test F1 scores over the SL baseline for both datasets. However, we found that algorithm efficacy depends heavily on the dataset’s loss landscape. On the sparse, flat landscape of the Adult dataset, GA’s population-based search was necessary to find well-generalizing optima (for F1 metric). Conversely, on the severely imbalanced Wine dataset, the greedy approach of RHC proved more effective at navigating deep local minima to improve minority class predictions. In Part 2, specific optimizers improved stability and convergence speed over the baseline, but in Part 3, regularization techniques failed to directly improve the test F1 score beyond the unregularized Adam baseline, though they offered computational and stability benefits.

Based on our budget-matched evidence, we establish a clear decision rule for optimizing the Adult dataset classification task. If standard adaptive methods like Adam stagnate, exhibit high cross-seed variance, or incur heavy computational overhead, practitioners should prioritize optimizer

settings by switching to simpler momentum-based methods. In our tests, SGD with momentum achieved superior stability with an IQR of 0.001 and faster wall-clock convergence at 8.6 seconds without sacrificing performance. Alternatively, removing momentum to create an RMSProp-like optimizer can stabilize Adam, which yielded the highest F1 score among adaptive variants.

When the optimizer overfits early, regularization should be the priority to improve compute efficiency and reduce variance. Early Stopping drastically reduces total training time, halting at 2,938 gradient evaluations instead of 11,300, with zero loss in test performance. If generalization variance across seeds is high, introducing moderate Noise Regularization, such as label smoothing and input noise, successfully narrowed our F1 IQR from 0.012 to 0.008. Finally, if gradient-based methods plateau and further performance improvement is critical, RO should be prioritized for final fine-tuning, as freezing the early layers and applying these search-based methods successfully pushed test metrics beyond the initial SL baseline. Regarding the limits of transfer, our findings are inherently coupled to the Adult dataset’s specific sparsity and architecture size, suggesting these guidelines may not directly apply to highly complex networks where techniques like AdamW and Dropout typically excel.

APPENDIX

AI Use Statement : As I tend to write verbosely, I used Gemini to enhance the conciseness and clarity of my writing and to correct spelling and grammatical errors on Overleaf. I also used GitHub Copilot in VS Code to assist with debugging and code generation. I carefully reviewed and understood all assisted content.

REFERENCES

- [1] Y. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio, “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization,” 2014. [Online]. Available: <https://arxiv.org/abs/1406.2572>
- [2] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [4] R. Müller, S. Kornblith, and G. Hinton, “When does label smoothing help?” 2020. [Online]. Available: <https://arxiv.org/abs/1906.02629>